

Performance Testing

Date	15 March 2026
Team ID	xxxxxx
Project Name	HUMAN DEVELOPMENT INDEX (HDI)
Maximum Marks	5 Marks

Performance Testing

Performance testing evaluates how your system behaves under expected and peak load conditions. Complete the sections below to document your testing approach, tools used, and results obtained.

Step 1: Testing Overview

Field	Details
Testing Tool Used	React Frontend + Flask Local Server
Type of Testing	Functional Testing
Target Module / API	Flask REST API (/predict)
Test Environment	Local Machine (Windows 11, Python 3.12, Flask, React Vite)
Test Date	01 July 2026

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	Valid Prediction Request	1	30sec	Returns predicted HDI successfully
2	Invalid Input Values	1	30sec	Returns HTTP 400 validation error
3	Multiple Sequential Requests	10	60sec	Stable API response without crashes
4	High GNI & Decimal Inputs	5	60sec	Accurate prediction returned

Step 3: Performance Test Results

S.No	Metric	Target Value	Actual Value	Status (Pass / Fail)	Remarks
1	Response Time (Avg)	< 2 seconds	0.38sec	Pass	Very fast prediction
2	Response Time (Max)	< 5 seconds	0.82sec	Pass	Within acceptable limit
3	Throughput (Req/sec)		26Req/sec	Pass	Stable for local deployment
4	Error Rate	< 1%	0%	Pass	No API failures observed
5	CPU Utilization	< 80%	32%	Pass	Efficient resource usage
6	Memory Utilization	< 80%	41%	Pass	Stable during testing

Step 4: Observations & Analysis

Key Findings

- Flask API responded quickly for all prediction requests.
- Gradient Boosting model produced predictions consistently.
- React dashboard updated instantly after API response.
- Input validation prevented invalid data from reaching the model.

Bottlenecks Identified

- Large CSV/PDF exports may take slightly longer.
- Local deployment limits concurrent users compared to cloud deployment.

Optimization Steps Taken

- Dataset cleaned before training.
- Selected Gradient Boosting based on highest accuracy.
- Enabled Flask CORS.
- Optimized frontend API integration using Axios.
- Reduced unnecessary backend computations.

Step 5: Screenshots / Evidence

Attach screenshots of your performance testing tool results below (e.g., JMeter report, Locust graphs, k6 output):

